Modélisation en Conception Orientée Objet

Pourquoi modéliser ?

- Le développement d'un logiciel est une tâche complexe et longue
- Plusieurs développeurs, plusieurs interlocuteurs, plusieurs technologies ...
- **Conséquence :** beaucoup de projets échouent, prennent du retard ou ne marche pas
- **Solution :** mise en place de techniques de **génie logiciel** (méthodes agiles, développement incrémentale ...)

Pourquoi modéliser ?

Étapes du développement d'un logiciel

1. Définition des besoins
2. Conception
3. Codage
4. Intégrations / tests

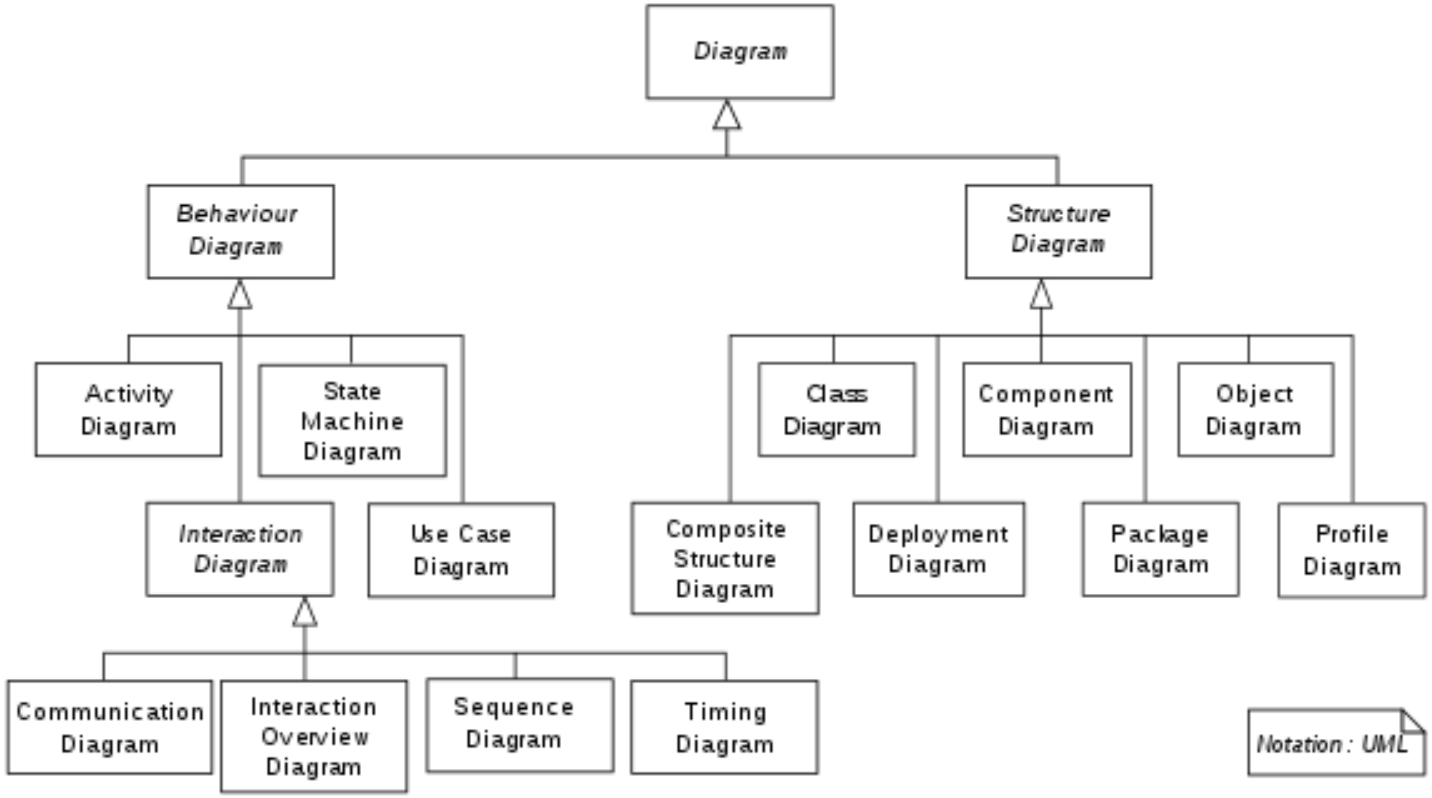
Besoin d'outils pour communiquer entre les différents acteurs / niveaux

Utilisation de modèles formels

Unified Modeling Language (UML)

Langage de modélisation définissant un ensemble de diagrammes pour modéliser :

- la définition des besoins
- la conception



Utilisation d'UML en pratique

- Des langages de programmation pour le codage proprement dit
- Orienté Objet : Java, C++, C# , **TypeScript** ...
- Des IDEs facilitant l'intégration et les tests

Programmation structurée

- Décomposition en sous-programmes (fonctions)
- Programme $\approx$ ensemble de sous-modules qui manipulent des données (variables, constantes, types ...)
- Séparation entre les données et le traitement
- Programmes grands ou complexes : problème de maintenance de code
- En effet, données et méthodes sont souvent étroitement liées
- Si on modifie les données, il faut souvent modifier le traitement et inversement

Programmation orientée objet

Objets = Réunion en une entité des données (attributs) et des traitements applicables (opérations, méthodes)

Classe = Description d'un objet $\approx$ type

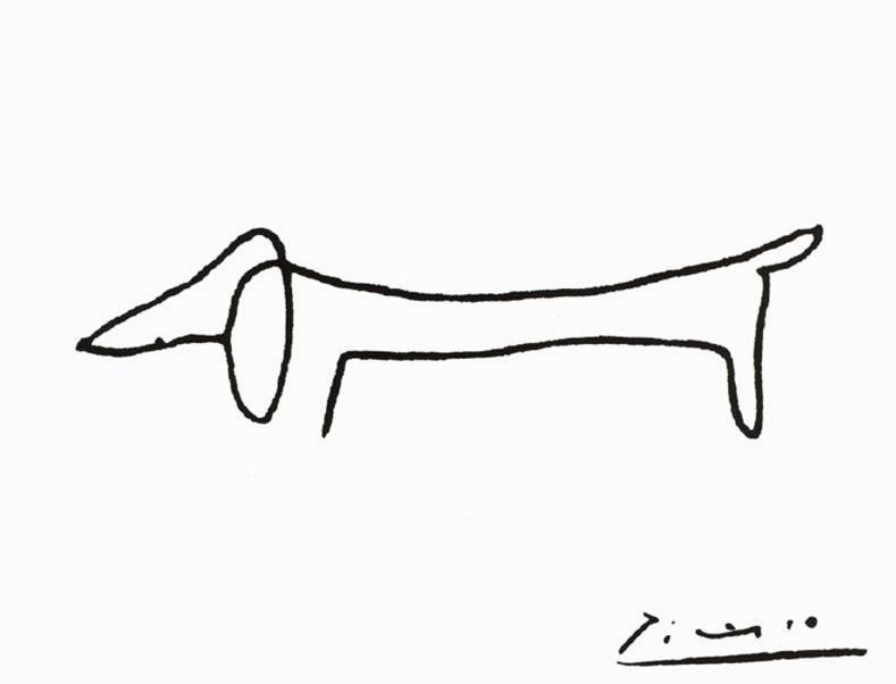
La classe Chien

Attributs

1. Race
2. Couleur de poil
3. Poids

Opérations

1. Le nourrir
2. Lui lancer une balle
3. Aller chercher la balle



Un objet : Raoul

Attributs

1. Pitbull
2. Poil blanc
3. 25 kg

Opérations

1. Le nourrir
2. Lui lancer une balle
3. Aller chercher la balle

Un autre objet : Hubert

Attributs

1. Cocker
2. Poil abricot
3. 12 kg

Opérations

1. Le nourrir
2. Lui lancer une balle
3. Aller chercher la balle

Classe et objet

Objet

- Regroupement en une entité d'information et de traitement
- Notion d'**encapsulation**
- Données : **attributs**, **champs**, **données**
- Fonctions / traitements : **méthodes**, **opérations**
- **Membres =** attributs + méthodes

Classe

- Définition d'un type lié à un objet
- $\approx$ une description (unique) d'un type
- Possibilité de déclarer des variables de ce type $=$ objet

Conception de classes : contrat et implémentation

- Répartition des membres de la classe en deux catégories :
  - **Publique :** accessible à tous
  - **Privé :** accessible uniquement aux membres de la classe
- **Encapsulation :** limiter (voire interdire) l'accès aux attributs = données
  - les attributs sont **privés** $\longrightarrow$ seul les méthodes de la classe peut y accéder
- Un attribut ne peut être manipuler que via les opérations de la classe
- Notion de **contrat** :
  i. En-têtes des méthodes **publiques** (accessibles à tous)
  ii. Comportement des opérations **publiques** (accessibles à tous)
- L'utilisateur n'a donc pas à connaître l'**implémentation** de la classe

Conception de classes : typologie des méthodes

1. Constructeurs : ce qu'il faut faire quand on crée un objet de la classe
2. Méthodes d'accès (en anglais *accessor*, en mauvais français *setter*)
    - Accès à la valeur d'un attribut sans modifier l'**état** de l'objet $\Leftrightarrow$ valeurs des attributs
3. Méthodes d'altération (en anglais *mutator*, en mauvais français *getter*)
    - Modification de la valeur d'un attribut de l'objet
4. Les autres (affichage, tri, sous-méthodes ...)

Définition d'une classe

- **Conception orientée objet :** diagramme de classe en UML
- **Programmation orientée objet :** définition d'un nouveau type (`class` en TypeScript)